UNIFIED MODELLING LANGUAGE

Fundamental Perspective

©PRODI. D4 T. TELEKOMUNIKASI, 2025

Agenda

- Pendahuluan
- Regenerasi UML 2.0
- Structural vs Behavioral
- Usecase Diagrams
- Latihan

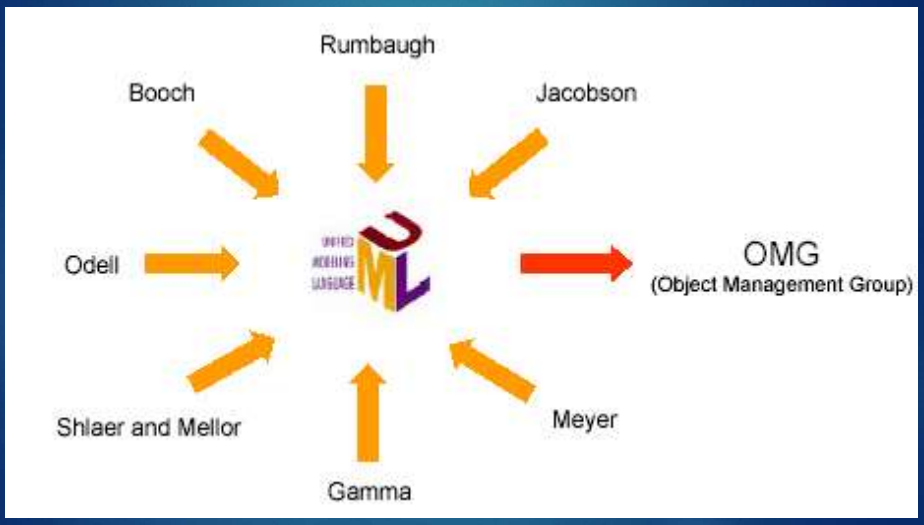
Pendahuluan

- Unified Modelling Language (UML) adalah sebuah "bahasa" yang telah menjadi standar dalam industri untuk visualisasi, merancang dan mendokumentasikan sistem informasi atau piranti lunak.
- UML menawarkan sebuah standar untuk merancang model sebuah sistem.
- Seperti bahasa-bahasa lainnya, UML mendefinisikan notasi dan *syntax*/semantik.

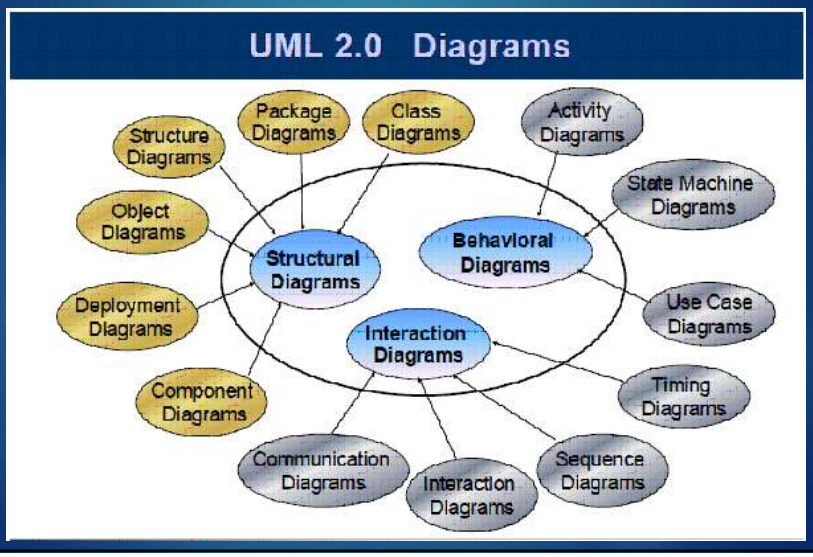
Pendahuluan (cont.)

- Notasi UML merupakan sekumpulan bentuk khusus untuk menggambarkan berbagai diagram piranti lunak.
- Setiap bentuk memiliki makna tertentu, dan UML syntax mendefinisikan bagaimana bentuk-bentuk tersebut dapat dikombinasikan.
- Notasi UML terutama diturunkan dari 3 notasi yang telah ada sebelumnya: Grady Booch OOD (Object-Oriented Design), Jim Rumbaugh OMT (Object Modeling Technique), dan Ivar Jacobson OOSE (Object-Oriented Software Engineering).

Regenerasi UML

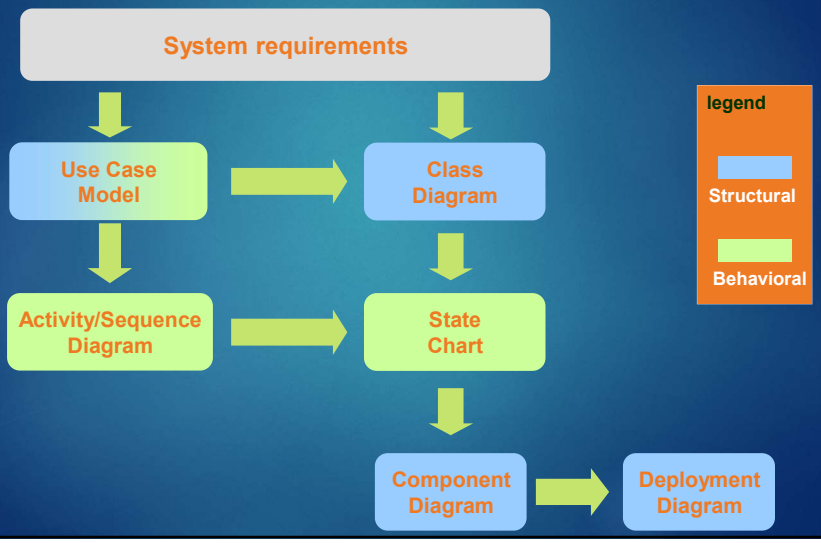


Regenerasi UML (cont.)



Regenerasi UML (cont.)

Analysis and Design Process By Zachman Framework



Structural **VS** Behavioral

- Diagram struktural dalam UML (Unified Modeling Language) menggambarkan struktur statis suatu sistem. Diagram ini menggambarkan komponen-komponen sistem dan hubungan antar komponennya. Diagram struktural digunakan untuk memvisualisasikan organisasi, komposisi, dan hubungan antar elemen yang berbeda dalam suatu sistem.

- Diagram perilaku dalam UML menggambarkan perilaku dinamis suatu sistem. Diagram ini menggambarkan bagaimana komponen-komponen suatu sistem berinteraksi satu sama lain dan bagaimana sistem tersebut merespons rangsangan eksternal. Diagram perilaku digunakan untuk memvisualisasikan perilaku suatu sistem dari waktu ke waktu.

Structural **VS** Behavioral (cont.)

Aspek	Diagram Struktural	Diagram Perilaku
Purpose	Menunjukkan struktur statis sistem	Mengilustrasikan perilaku dinamis sistem
Focus	Fokus pada komponen, kelas, dan hubungannya	Fokus pada interaksi antara komponen dan kelas
Elements	Kelas, objek, antarmuka, komponen, dan hubungannya	Aktivitas, status, pesan, peristiwa, dan interaksinya
Representation	Penggunaan class diagrams, object diagrams, component diagrams, dll.	Penggunaan activity diagrams, state machine diagrams, sequence diagrams, dll.
Time	Tidak tergantung waktu, karena menggambarkan struktur pada waktu tertentu	Bergantung waktu, karena menggambarkan bagaimana sistem berperilaku seiring berjalannya waktu
Example Use Cases	Class Diagram untuk memodelkan struktur sistem perangkat lunak	sequence diagram untuk menggambarkan interaksi antara objek dalam skenario tertentu

https://www.geeksforgeeks.org/difference-between-structural-diagrams-and-behavioral-diagrams/

Use case diagram

- Use case diagram menggambarkan fungsionalitas yang diharapkan dari sebuah sistem. Yang ditekankan adalah “apa” yang diperbuat sistem, dan bukan “bagaimana”.
- Sebuah use case merepresentasikan sebuah interaksi antara aktor dengan sistem.

USE CASE



- Use case dibuat berdasar keperluan actor, merupakan "apa" yang dikerjakan system, bukan "bagaimana" system mengerjakannya
- Use case diberi nama yang menyatakan apa hal yang dicapai dari hasil interaksinya dengan actor.
- *Use case* dinotasikan dengan gambar (horizontal ellipse)
- Use case biasanya menggunakan kata kerja
- Nama use case boleh terdiri dari beberapa kata dan tidak boleh ada 2 use case yang memiliki nama yang sama

ACTOR



- Actor menggambarkan orang, system atau external entitas / stakeholder yang menyediakan atau menerima informasi dari system
- Actor menggambarkan sebuah tugas/peran dan bukannya posisi sebuah jabatan
- Actor memberi input atau menerima informasi dari system
- Actor biasanya menggunakan Kata benda
- Tidak boleh ada komunikasi langsung antar actor
- Indikasi <<system>> untuk sebuah actor yang merupakan sebuah system
- Adanya actor bernama "Time" yang mengindikasikan scheduled events (suatu kejadian yang terjadi secara periodik/bulanan)
- Letakkan actor utama anda pada pojok kiri atas dari diagram

Association

- Associations bukan menggambarkan aliran data/informasi
- Associations digunakan untuk menggambarkan bagaimana actor terlibat dalam use case
- Ada 4 jenis relasi yang bisa timbul pada use case diagram
  1. Association antara actor dan use case
  2. Association antara use case
  3. Generalization/Inheritance antara use case
  4. Generalization/Inheritance antara actors

Association antara actor dan use case

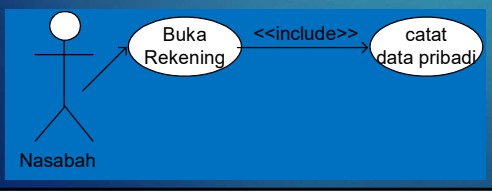
- Ujung panah pada association antara actor dan use case mengindikasikan ***siapa/apa*** yang meminta interaksi dan bukannya mengindikasikan aliran data
- **Sebaiknya** gunakan **Garis tanpa panah** untuk association antara actor dan use case

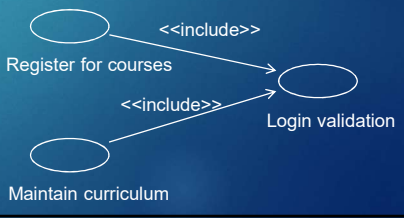
- association antara actor dan use case yang menggunakan **panah terbuka** untuk mengindikasikan bila actor berinteraksi secara ***pasif*** dengan system anda



Association antara use case

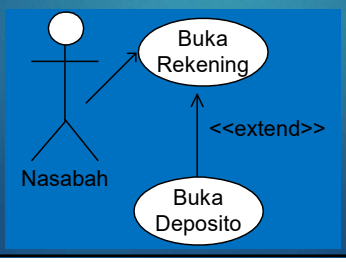
- <<include>> termasuk didalam use case lain (required) / (diharuskan)
  - Pemanggilan use case oleh use case lain, contohnya adalah pemanggilan sebuah fungsi program
  - Tanda panah terbuka harus terarah ke sub use case
  - Gambarkan association include secara horizontal





Association antara use case (Lanjut)

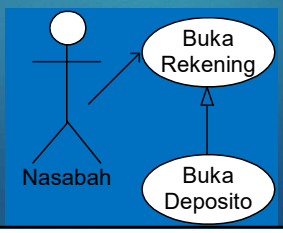
- <<extend>> perluasan dari use case lain jika kondisi atau syarat terpenuhi
  - Kurangi penggunaan association Extend ini, terlalu banyak pemakaian association ini membuat diagram sulit dipahami.
  - Tanda panah terbuka harus terarah ke parent/base use case
  - Gambarkan association extend secara vertical



Generalization/inheritance antara use case

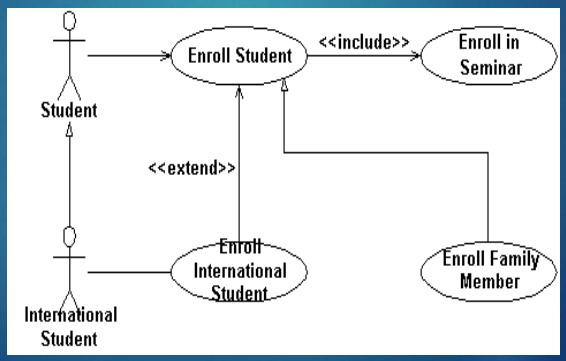
- Generalization/inheritance digambarkan dengan sebuah garis berpanah tertutup pada salah satu ujungnya yang menunjukkan lebih umum

- Gambaran generalization/inheritance antara use case secara vertical dengan inheriting use case dibawah base/parent use case
- Generalization/inheritance dipakai ketika ada sebuah keadaan yang lain sendiri/perlakuan khusus *(single condition)*



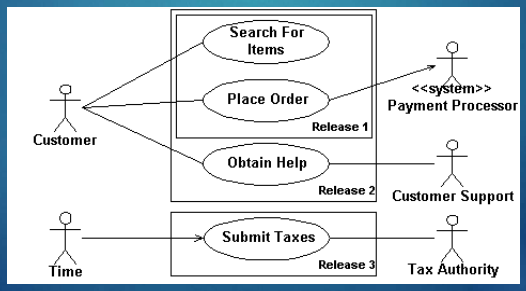
Generalization/inheritance antara actor

- Gambaran generalization/inheritance antara actors secara vertical dengan inheriting actor dibawah base/parent use case



Use case System boundary boxes

- Digambarkan dengan kotak disekitar use case, untuk menggambarkan jangkauan system anda (scope of your system).
- Biasanya digunakan apabila memberikan beberapa alternative system yang dapat dijadikan pilihan
- System boundary boxes dalam penggunaannya optional



Latihan Use Case Diagram !

PT. Nusantara adalah sebuah perusahaan yang bergerak dibidang penjualan Tunai barang-barang elektronik. Semua transaksi di perusahaan masih dilakukan secara manual. Berikut ini adalah kegiatan kegiatan yang dilakukan oleh **bagian Penjualan** dalam melaksanakan transaksi penjualan Barang di dalam perusahaan.

1. Pemesanan barang

Setiap kali Bagian penjualan akan menjual barang ia selalu menerima **surat pesanan** dari **pelanggan**. Berdasarkan Surat pesanan tersebut bagian penjualan kemudian mencatat dan merekamnya kedalam Arsip Surat Pesanan. Berdasarkan Arsip surat pesanan tersebut, bagian penjualan membuatkan **Faktur** dan **Surat Jalan** yang dikirimkan kepada **Pelanggan** sebagai bukti bahwa barang yang dipesan sudah terealisasi dan rangkapnya disimpan sebagai Arsip Faktur dan Arsip Surat Jalan.

Latihan Use Case Diagram (cont.)

2. Pembuatan Kwitansi

Apabila **Faktur** dan Surat Jalan sudah sampai ditempat pelanggan, maka pelanggan megirimkan Pembayaran yang kemudian oleh **bagian penjualan** dibuatkan **Kwitansi** yang dibuat berdasarkan Arsip Faktur yang kemudian diserahkan kepada **pelanggan** sebagai bukti pembayaran dan rangkapnya disimpan kedalam Arsip Kwitans

3. Pembuatan Laporan

Setiap akhir bulan **Bagian Penjualan** selalu membuat **Laporan Penjualan** berdasarkan Arsip Faktur dan **Laporan Pesanan** berdasarkan Arsip Pesanan dan **Laporan Pengiriman** berdasarkan Arsip Surat Jalan yang ditujukan kepada **Kepala Bagian Penjualan**

Diminta :

- Buatlah Use case diagram dari data diatas !

